

AC Power Meter V.0

Technical Report

Jadavpur University

Department of Electrical Engineering

Team Members:

Dwaipayan Shikari

Shramana Mondal

Prattay Barua

Jayrup Das

Repository:

<https://github.com/dwan6767/UNIOPME>

Raspberry Pi Pico · MicroPython · Open Source Hardware

1. Project Overview

The **AC Power Meter V.0** is a low-cost, open-source hardware instrument for measuring AC electrical parameters, developed as an academic project at the **Department of Electrical Engineering, Jadavpur University**. It uses a Raspberry Pi Pico running MicroPython to sample current signals via the ZMCT103C sensor, then computes **RMS current**, **line frequency**, and **Total Harmonic Distortion (THD)** in real time. Results are output over the serial REPL at 500 ms intervals.

Note — Voltage measurement: The PCB uses an RP2040 (QFN-56 package) soldered directly onto the board. Due to soldering difficulties with the fine-pitch QFN package, the analogue pin (GPIO 26) for the ZMPT101B voltage sensor is not fully connected. Consequently, only **current-based measurements** (I_{RMS} , frequency from current signal, THD from current signal) are functional in this prototype. The voltage sensor channel is wired identically to the current channel in the firmware, so once the hardware is reworked the full two-channel measurement will operate.

Property	Value
Microcontroller	Raspberry Pi Pico (RP2040)
Firmware Language	MicroPython
ADC Resolution	16-bit (0–3.3 V)
Sampling Rate	5 kHz (500 samples per window)
Measured Quantities	I_{RMS} , f , THD
License	CERN Open Hardware License v2.0 (CERN-OHL-P)
Repository	github.com/dwan6767/UNIOPME

2. Theoretical Background

2.1 Alternating Current (AC) Fundamentals

An alternating current waveform varies sinusoidally with time:

$$i(t) = I_{pk} \sin(2\pi f t + \varphi)$$

where I_{pk} is the peak current, f is the frequency (50 or 60 Hz for mains), and φ is the phase angle. The **RMS** (Root Mean Square) value is the equivalent DC current that would dissipate the same power in a resistive load:

$$I_{RMS} = \sqrt{(1/T) \int_0^T i(t)^2 dt}$$

For a pure sine wave, $I_{RMS} = I_{pk} / \sqrt{2}$. However, in practice the waveform contains harmonics, making direct digital sampling and numerical RMS computation necessary.

2.2 Discrete Sampling Theory

The Raspberry Pi Pico's ADC converts the continuous analogue signal into a discrete time series. The **Nyquist-Shannon sampling theorem** states that to faithfully reconstruct a signal, the sampling frequency f_s must be at least twice the highest frequency component present:

$$f_s \geq 2 f_{max}$$

This project samples at 5 kHz ($T_s = 200 \mu s$), allowing analysis of harmonics up to 2.5 kHz (the 50th harmonic of 50 Hz mains). A window of $N = 500$ samples captures 10 full cycles of a 50 Hz waveform, providing good frequency resolution ($\Delta f = f_s/N = 10$ Hz).

2.3 DC Offset Removal

The ADC output contains a DC bias due to the sensor's quiescent output voltage. The DC offset is estimated as the arithmetic mean of the sample window and subtracted from each sample:

$$\begin{aligned} offset &= (1/N) \sum_{k=0}^{N-1} x[k] \\ s[k] &= x[k] - offset \end{aligned}$$

2.4 RMS Computation in Discrete Time

For a discrete signal $s[k]$ of length N , the RMS is:

$$s_{RMS} = \sqrt{(1/N) \sum_{k=0}^{N-1} s[k]^2}$$

This is exact for periodic signals when N spans an integer number of cycles and the sampling is uniform.

2.5 Frequency Detection via Zero-Crossing

Zero-crossing detection measures the number of times the AC waveform crosses its mean (zero after offset removal) in the positive-going direction. For a window of N samples:

$$f = f_s \times N_{crossings} / (N - 1)$$

The frequency resolution improves with longer observation windows. A minimum of two crossings (≥ 1 full cycle) is required for a valid reading.

2.6 Total Harmonic Distortion (THD)

THD quantifies how much the waveform deviates from a pure sinusoid. It is defined as the ratio of the RMS of all harmonic components to the RMS of the fundamental:

$$THD = \sqrt{(\sum_{h=2}^{\infty} H_h^2)} / H_1 \times 100 [\%]$$

where H_1 is the fundamental amplitude and H_h are the harmonic amplitudes. This implementation uses a **simplified DFT** (often called the Goertzel algorithm) evaluating the complex Fourier coefficient at the fundamental frequency and at harmonics 2 through 10:

$$X[h] = \sum_{n=0}^{N-1} s[n] \cdot (\cos(2\pi h n / N) - j \sin(2\pi h n / N))$$

$$|H_h| = 2 \times |X[h]| / N$$

The $2/N$ scaling accounts for the fact that the DFT gives the full peak-to-peak amplitude when the frequency aligns exactly with a bin centre. Harmonics up to H_{10} are computed; higher-order contributions are typically negligible for mains waveforms.

3. Hardware Design

3.1 Components

Component	Role	ADC Pin	Status
Raspberry Pi Pico	Microcontroller & ADC	—	OK
ZMPT101B	AC Voltage Sensor	GPIO 26	Not connected (soldering defect)
ZMCT103C	AC Current Sensor	GPIO 27	Functional

3.2 Sensor Details

ZMPT101B — Voltage Sensor

The ZMPT101B is a PCB-mount voltage transformer based on a precision ZMPT101B micro voltage transformer. It provides galvanic isolation between the mains and the measurement circuit. The output is an AC voltage proportional to the input, biased at $V_{CC}/2$ for single-supply ADC interfacing. Rated for up to 250 V RMS input with a transformation ratio of 1000:1000 (1:1).

ZMCT103C — Current Sensor

The ZMCT103C is a precision current transformer with a built-in burden resistor. It outputs an AC voltage linearly proportional to the primary current. Typical sensitivity: 1 V per 5 A input. The output is biased at $V_{CC}/2$ and directly readable by the Pico ADC. Used here for the current measurement channel (GPIO 27).

3.3 RP2040 Soldering Issue

Known hardware limitation: The RP2040 is packaged as a QFN-56 (7×7 mm, 0.4 mm pitch). Hand-soldering this fine-pitch package on a two-layer PCB proved challenging. The GPIO 26 pad (voltage sensor ADC input) did not form a reliable solder joint. All other pins were verified functional. A rework with hot-air soldering or a professionally-assembled board will restore the voltage

measurement channel. The firmware already supports both channels; only ADC pin initialisation for GPIO 26 is required.

3.4 Schematic

The full schematic and EasyEDA project are available in the `Designs/` directory:

- `Designs/schematic.pdf` — Circuit schematic
- `Designs/rp.eprj` — EasyEDA project file
- `Designs/power.zip` — Gerber fabrication files

3.5 Enclosure

3D-printable enclosure designs in PDF format are available under `Models/` (files `1_p.pdf` and `2_p.pdf`). Renders of the assembled model are shown in the Gallery section.

4. Firmware

4.1 Signal Processing Pipeline

1. **Sampling** — 500 ADC readings at 200 μs intervals (5 kHz).
2. **DC offset removal** — The arithmetic mean of the buffer is subtracted from each sample.
3. **Scaling** — Raw ADC counts are converted to volts, then multiplied by the current calibration factor ICAL.
4. **RMS calculation** — Square root of the mean of squared samples.
5. **Frequency detection** — Zero-crossing count divided by the window duration.
6. **THD calculation** — Discrete Fourier analysis of the fundamental and harmonics 2-10 via the Goertzel algorithm.

4.2 Calibration

Parameter	Value	Description
ICAL	$16.667 / 3 \approx 5.556$	Current sensor sensitivity & burden scaling
SAMPLES	500	Samples per measurement window
SAMPLE_DELAY_US	200 μs	Interval between successive samples
ADC_VREF	3.3 V	ADC reference voltage
ADC_COUNTS	65 535	16-bit ADC full-scale count ($2^{16} - 1$)

4.3 Source Code

The complete firmware (`Firmware/main.py`, 99 lines):

```
import utime
import math
from machine import ADC, Pin

SAMPLES = 500
SAMPLE_DELAY_US = 200
ADC_VREF = 3.3
ADC_COUNTS = 65535
ICAL = 16.667 / 3
```

```

adc_i = ADC(Pin(27))

def print_header():
    print(r"""
=====

  |  _  \ / _  \ \      / /  _  |  _  \
  | |_) | | | \ \ ^ / / | | | |_) |
  |  _ / | | | \ \ ^ / / |  _ | |  _ /
  | |  | | | | \ \ ^ / | | | | \ \
  | |  \ _ /  \ \ ^ / | | | | \ \

  |  \ |  _  |  _  |  _  |  _  \
  | \ / | |  | | | | | | |_) |
  | | \ | |  | | | | | |  _ /
  | | | | |  | | | | | | \ \
  | | | | |  | | | | | | \ \

                                POWER METER V.1
=====

""")

def capture_samples():
    raw_sum = 0
    buffer = []
    for _ in range(SAMPLES):
        val = adc_i.read_u16()
        buffer.append(val)
        raw_sum += val
    utime.sleep_us(SAMPLE_DELAY_US)
    offset = raw_sum / SAMPLES
    scaled = [(v - offset) * (ADC_VREF / ADC_COUNTS) * ICAL for v in buffer]
    return scaled

def calc_rms(samples):
    sum_sq = sum(s * s for s in samples)
    return math.sqrt(sum_sq / len(samples))

def calc_frequency(samples):
    crossings = 0
    offset = sum(samples) / len(samples)
    for k in range(1, len(samples)):
        if (samples[k - 1] - offset) <= 0 and (samples[k] - offset) > 0:
            crossings += 1
    if crossings < 2:
        return 0.0
    sample_rate = 1_000_000 / SAMPLE_DELAY_US
    return sample_rate * crossings / (SAMPLES - 1)

def calc_thd(samples):
    N = len(samples)
    fund_real = 0.0
    fund_imag = 0.0
    harm_power = 0.0
    for n in range(N):
        angle = 2 * math.pi * n / N
        fund_real += samples[n] * math.cos(angle)
        fund_imag += samples[n] * math.sin(angle)

```



```

fund_mag = 2 * math.sqrt(fund_real ** 2 + fund_imag ** 2) / N
for h in range(2, 11):
    h_real = 0.0
    h_imag = 0.0
    for n in range(N):
        angle = 2 * math.pi * h * n / N
        h_real += samples[n] * math.cos(angle)
        h_imag += samples[n] * math.sin(angle)
    h_mag = 2 * math.sqrt(h_real ** 2 + h_imag ** 2) / N
    harm_power += h_mag ** 2
return math.sqrt(harm_power) / fund_mag * 100 if fund_mag > 0 else 0.0

print_header()
utime.sleep(1)
print("Starting Current, Frequency & THD Measurements...")

while True:
    try:
        samples = capture_samples()
        irms = calc_rms(samples)
        freq = calc_frequency(samples)
        thd = calc_thd(samples)
        print("-" * 40)
        print(f"Current:  {irms:6.3f} A")
        print(f"Freq:      {freq:6.2f} Hz")
        print(f"THD:        {thd:6.2f} %")
        utime.sleep_ms(500)
    except Exception as e:
        print(f"Error: {e}")
        utime.sleep_ms(2000)

```

5. Measurement Algorithms — Derivation

5.1 RMS Current

Let $s[k]$ be the k -th sample after offset removal and scaling, representing instantaneous current in amperes. The RMS estimate over N samples is:

$$I_{RMS} = \sqrt{(1/N) \sum_{k=0}^{N-1} s[k]^2}$$

This formula is the discrete-time equivalent of the continuous RMS integral. With $N = 500$ and $f_s = 5$ kHz, the window spans 100 ms, containing exactly 5 cycles of a 50 Hz (or 6 cycles of a 60 Hz) mains waveform, giving a stable reading.

5.2 Frequency via Zero-Crossing

After DC offset removal, the signal oscillates around zero. Each positive-going transition (sample $k-1 \leq 0$ and sample $k > 0$) represents one half-cycle. The total number of zero crossings divided by the window duration gives the frequency. With N crossings counted over N samples at sample period T_s :

$$f = (N_{crossings} \times f_s) / (N - 1)$$

The denominator $N-1$ accounts for the number of intervals between N samples. The minimum detectable frequency is $2f_s/N = 20$ Hz (at 2 crossings), safely below the 50/60 Hz mains range.

5.3 Total Harmonic Distortion via DFT

The discrete Fourier transform (DFT) at bin h is:

$$X[h] = \sum_{n=0}^{N-1} s[n] \cdot e^{-j2\pi h n / N}$$

The magnitude of the h -th harmonic is:

$$|H_h| = (2/N) \times |X[h]|$$

The factor $2/N$ converts from DFT magnitude to peak amplitude of the sinusoidal component (the DFT distributes the signal energy between positive and negative frequency bins). THD is then:

$$THD = \sqrt{(\sum_{h=2}^{10} |H_h|^2)} / |H_1| \times 100 [\%]$$

Only harmonics up to the 10th are evaluated. Since the sampling rate (5 kHz) can represent harmonics up to the 49th (2.45 kHz at 50 Hz fundamental), contributions from H_{11} through H_{49} are assumed negligible for typical mains loads — a common assumption in power quality measurements per IEC 61000-4-7.

5.4 Computational Complexity

A full N-point FFT requires $O(N \log N)$ operations, but computing only 10 specific frequency bins (fundamental + 9 harmonics) via the direct DFT (Goertzel) approach uses $O(10N)$ operations. For $N = 500$, this is approximately 5000 multiply-adds per measurement cycle, easily within the RP2040's capabilities at 133 MHz.

6. Gallery

3D Render (Front)



3D Render (Rear)



3D Render (Angled)



2D Model



Test Setup



Test with load



7. Project Structure

```
Hardware/
+-- Designs/          # Schematic (schematic.pdf)
|   +-- rp.eprj       # EasyEDA project
|   +-- power.zip     # Gerber files
|   +-- schematic.pdf
+-- Docs/
|   +-- AC_Power_Meter_Report.docx
|   +-- POWER METER-2.pdf
+-- Firmware/
|   +-- main.py       # MicroPython source (99 lines)
+-- Models/           # 3D renders, photos, enclosure PDFs
|   +-- 0.png
|   +-- 1.png / 1_p.pdf
|   +-- 2.png / 2_p.pdf
|   +-- 3.png
|   +-- 2d_model.jpeg
|   +-- test_setup.jpeg
|   +-- test_soft.jpeg
|   +-- test.jpeg
+-- report.html       # This file
+-- README.md
```

8. License

This project is certified [Open Source Hardware](#) under the [CERN Open Hardware License v2.0](#) (CERN-OHL-P).